

```
import numpy as np

[1] ✓ 2.7s

class Logistic_Regression():
    def __init__(self, learning_rate , no_of_iterations):
        self.learning_rate = learning_rate
        self.no_of_iterations = no_of_iterations

    def fit(self, X, Y):
        self.m, self.n = X.shape

        self.w = np.zeros(self.n)
        self.b = 0
        self.X = X
        self.Y = Y

        for i in range(self.no_of_iterations):
            self.update_weights()

    def update_weights(self):
        Y_hat = 1 / (1+np.exp(-(self.X.dot(self.w) + self.b)))
        dw = (1 / self.m) * np.dot(self.X.T, (Y_hat - self.Y))
        db = (1 / self.m) * np.sum(Y_hat - self.Y)

        self.w = self.w - self.learning_rate * dw
        self.b = self.b - self.learning_rate * db

    def predict(self, X):
        Y_pred = 1 / (1+np.exp(-(X.dot(self.w) + self.b)))
        Y_pred = np.where(Y_pred > 0.5, 1, 0)
        return Y_pred

[2] ✓ 0.0s

import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

[3] ✓ 11.7s
```

```
diabetes_dataset = pd.read_csv('/Users/panesh/coding2/MachineLearningFrom0/datasets/diabetes.csv')

[4] ✓ 0.1s

diabetes_dataset.head()

[5] ✓ 0.0s
..
  Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  DiabetesPedigreeFunction  Age  Outcome
0           6     148             72           35         0   33.6              0.627      50         1
1           1      85             66           29         0   26.6              0.351      31         0
2           8     183             64            0         0   23.3              0.672      32         1
3           1      89             66           23        94   28.1              0.167      21         0
4           0     137             40           35       168   43.1              2.288      33         1

diabetes_dataset.shape

[6] ✓ 0.0s
..
(768, 9)

diabetes_dataset.describe()

[7] ✓ 0.0s
..
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  DiabetesPedigreeFunction  Age  Outcome
count  768.000000  768.000000  768.000000  768.000000  768.000000  768.000000  768.000000  768.000000  768.000000
mean     3.845052  120.894531   69.105469   20.536458   79.799479   31.992578   0.471876   33.240885   0.348958
std     3.369578   31.972618   19.355807   15.952218  115.244002   7.884160   0.331329   11.760232   0.476951
min      0.000000   0.000000   0.000000   0.000000   0.000000   0.000000   0.078000   21.000000   0.000000
25%      1.000000   99.000000   62.000000   0.000000   0.000000   27.300000   0.243750   24.000000   0.000000
50%      3.000000  117.000000   72.000000  23.000000   30.500000   32.000000   0.372500   29.000000   0.000000
75%      6.000000  140.250000   80.000000  32.000000  127.250000   36.600000   0.626250   41.000000   1.000000
max     17.000000  199.000000  122.000000  99.000000  846.000000  67.100000   2.420000   81.000000   1.000000

diabetes_dataset['Outcome'].value_counts()

[8] ✓ 0.0s
..
Outcome
0     500
1     268
```

```
features = diabetes_dataset.drop(columns='Outcome', axis=1)
target = diabetes_dataset['Outcome']

[9] ✓ 0.0s

print(features)

[10] ✓ 0.0s

...
Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin  BMI  \
0           6      148           72           35         0  33.6
1           1       85           66           29         0  26.6
2           8     183           64           0         0  23.3
3           1       89           66           23        94  28.1
4           0     137           40           35       168  43.1
..         ...     ...           ...           ...     ...   ...
763         10     101           76           48       180  32.9
764          2     122           70           27         0  36.8
765          5     121           72           23       112  26.2
766          1     126           60           0         0  30.1
767          1       93           70           31         0  30.4

DiabetesPedigreeFunction  Age
0           0.627      50
1           0.351      31
2           0.672      32
3           0.167      21
4           2.288      33
..         ...     ...
763         0.171      63
764         0.340      27
765         0.245      30
766         0.349      47
767         0.315      23

[768 rows x 8 columns]

scaler = StandardScaler()

[11] ✓ 0.0s
```

```
scaler.fit(features)

[12] ✓ 0.0s

...
StandardScaler ⓘ ⓘ
StandardScaler()

standardized_data = scaler.transform(features)

[13] ✓ 0.0s

print(standardized_data)

[14] ✓ 0.0s

...
[[ 0.63994726  0.84832379  0.14964075 ...  0.20401277  0.46849198
  1.4259954 ]
 [-0.84488505 -1.12339636 -0.16054575 ... -0.68442195 -0.36506078
 -0.19067191]
 [ 1.23388019  1.94372388 -0.26394125 ... -1.10325546  0.60439732
 -0.10558415]
 ...
 [ 0.3429808  0.00330087  0.14964075 ... -0.73518964 -0.68519336
 -0.27575966]
 [-0.84488505  0.1597866  -0.47073225 ... -0.24020459 -0.37110101
  1.17073215]
 [-0.84488505 -0.8730192  0.04624525 ... -0.20212881 -0.47378505
 -0.87137393]]

features = standardized_data
target = diabetes_dataset['Outcome']

[15] ✓ 0.0s
```

```
print(features)
print(target)
[16] ✓ 0.0s
... [[ 0.63994726  0.84832379  0.14964075 ... 0.20401277  0.46849198
1.4259954 ]
[-0.84488505 -1.12339636 -0.16054575 ... -0.68442195 -0.36506078
-0.19067191]
[ 1.23388019  1.94372388 -0.26394125 ... -1.10325546  0.60439732
-0.10558415]
...
[ 0.3429808  0.00330087  0.14964075 ... -0.73518964 -0.68519336
-0.27575966]
[-0.84488505  0.1597866 -0.47073225 ... -0.24020459 -0.37110101
1.17073215]
[-0.84488505 -0.8730192  0.04624525 ... -0.20212881 -0.47378505
-0.87137393]]
0 1
1 0
2 1
3 0
4 1
..
763 0
764 0
765 0
766 1
767 0
Name: Outcome, Length: 768, dtype: int64

X_train, X_test, Y_train, Y_test = train_test_split(features, target, test_size = 0.2,)
[17] ✓ 0.0s

print(features.shape, X_train.shape, X_test.shape)
[18] ✓ 0.0s
... (768, 8) (614, 8) (154, 8)

classifier = LogisticRegression(learning_rate=0.01, no_of_iterations=1000)
[19] ✓ 0.0s
```

```
classifier.fit(X_train, Y_train)
[20] ✓ 0.0s

X_train_prediction = classifier.predict(X_train)
training_data_accuracy = accuracy_score(Y_train, X_train_prediction)
[21] ✓ 0.0s

print('accuracy score of the training data', training_data_accuracy)
[22] ✓ 0.0s
... accuracy score of the training data 0.7801302931596091

input_data = (5, 166, 72, 19, 175, 25.8, 0.587, 51)
[23] ✓ 0.0s

input_data_as_numpy_array = np.array(input_data)
[24] ✓ 0.0s

input_data_resaped = input_data_as_numpy_array.reshape(1,-1)
std_data = scaler.transform(input_data_resaped)
print(std_data)
prediction = classifier.predict(std_data)
print(prediction)
if(prediction[0]==0):
    print('The person is not diabetic')
else:
    print('The person is diabetic')
[25] ✓ 0.0s
... [[ 0.3429808  1.41167241  0.14964075 -0.09637905  0.82661621 -0.78595734
0.34768723  1.51108316]]
[1]
The person is diabetic
```